

Grafos – Estruturas de Dados para representar Grafos

Teoria dos Grafos – 2024-2

Prof. Roberto C. de Araujo

1. Introdução

Os grafos fornecem um modelo bastante útil para representar problemas importantes do mundo real. Por isso, é fundamental fornecer:

- Estruturas de dados adequadas para representar computacionalmente os grafos, e
- Algoritmos para resolver problemas envolvendo grafos.

Neste texto elaboramos alguns algoritmos básicos para tratar de grafos e apresentamos três propostas de estruturas de dados para representar os grafos.

2. Algoritmo para decidir a conexidade de grafo

Apesar de existirem algoritmos eficientes e elegantes para decidir o problema da conexidade (e que veremos posteriormente), exibimos neste momento um algoritmo mais simples e de fácil compreensão:

```
/*  
 * Recebe um grafo G não vazio como  
 * argumento e retorna:  
 * true, se o grafo for conexo; ou  
 * false, se o grafo não for conexo.  
 */
```

Bool eConexo (G, ordemG)

1. Escolha um vértice v_0 de G e marque-o com 1.
2. Todos os demais vértices do grafo serão marcados com 0.

3. Repita

Se existir uma aresta que incide em um vértice v_i marcado com 1 e, também, em um vértice v_j , marcado com 0, marque v_j com 1.

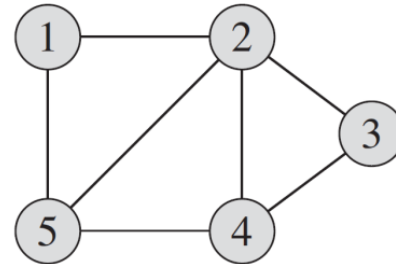
até que nenhum novo vértice seja marcado com 1.

4. Se todos os vértices estiverem marcados com 1, retorne **true**; caso contrário retorne **false**.

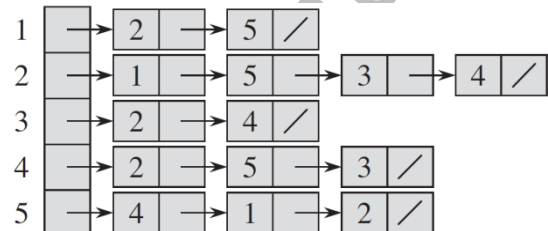
Note que o algoritmo acima não depende de uma representação computacional específica do grafo, mas, para implementá-lo, deveremos considerar uma estrutura de dados adequada.

3. Estruturas de Dados para Grafos

Considere o grafo desenhado abaixo. Vamos construir duas estruturas de dados diferentes para ele.



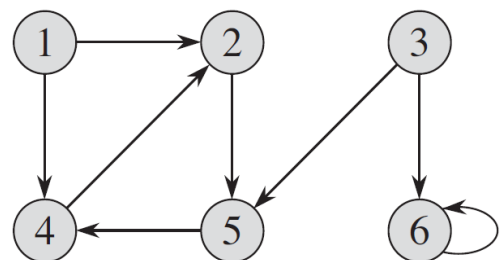
Representação por **lista de adjacência**:



Representação por **matriz de adjacência**:

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

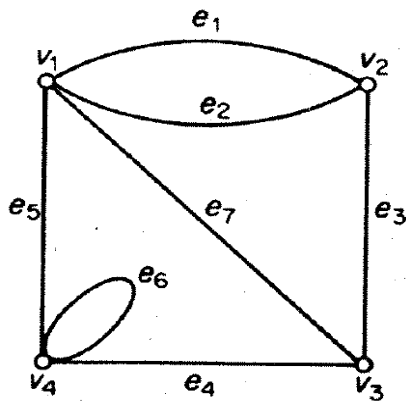
Representação de **grafos orientados**:



Representação por lista de adjacência e por matriz de adjacência:

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Uma outra forma de representar um grafo é a **matriz de incidência**. Abaixo, temos um grafo G e suas representações por matriz de incidência $M(G)$ e por matriz de adjacência $A(G)$.



	e_1	e_2	e_3	e_4	e_5	e_6	e_7		v_1	v_2	v_3	v_4
v_1	1	1	0	0	1	0	1	v_1	0	2	1	1
v_2	1	1	1	0	0	0	0	v_2	2	0	1	0
v_3	0	0	1	1	0	0	1	v_3	1	1	0	1
v_4	0	0	0	1	1	2	0	v_4	1	0	1	1

$M(G)$

$A(G)$

4. Exercícios

1. Ajuste o algoritmo que decide se um grafo é conexo ou não para que ele calcule a quantidade de componentes do grafo.
2. Ajuste o algoritmo que decide se um grafo é conexo ou não para que ele calcule a distância entre um vértice inicial s (passado como argumento) e todos os demais vértices do grafo.
3. Elabore um algoritmo para calcular o grau de um determinado vértice de um grafo.
4. Dado um grafo H , elabore um algoritmo para calcular $\delta(H)$.
5. Dado um grafo H , elabore um algoritmo para calcular $\Delta(H)$.
6. Dado um grafo H , elabore um algoritmo para decidir se H é um grafo regular ou não.
7. Dado um grafo H , elabore um algoritmo para decidir se H é um grafo simples ou não.

8. Elabore um algoritmo para decidir se um determinado vértice v de um grafo é um vértice de corte ou não.
9. Elabore um algoritmo para decidir se uma determinada aresta a de um grafo é uma aresta de corte ou não.

5. Bibliografia

BONDY, A., MURTY, U.S.R. **Graph Theory**. New York: Springer, 2010.

CORMEN, T. H. **Introduction to algorithms**. 3rd ed. Cambridge, MA: MIT Press, 2009.